ChefKart: Weeks 6 and 7

Completed practicals with simple code explanations. Updated 13 October 2026. Open the extracted **ChefKart** folder in VS Code before running commands.

1. Corrected aims for your project

Week 6(a) - Cooking service catalogue. Design a MongoDB database for ChefKart containing service ID, service title, chef name, category, price, and availability. Write MongoDB commands to insert multiple cooking services, retrieve available services, update service availability, and delete a service.

Week 6(b) - Node.js cook records. Develop a Node.js program to establish a MongoDB connection and perform insert, retrieve, update, and delete operations on a collection of ChefKart cook records containing chef ID, name, cuisine, city, visit price, and availability.

Week 7(a), Exercise 1 - Cook details form. Develop a Python web application for ChefKart using Flask. Accept the cook's name and cuisine or specialization through an HTML form. Validate the input and display the entered cook details.

Week 7(a), Exercise 2 - Cooking appointment form. Develop a Python web application for ChefKart using Flask. Accept customer name, preferred cook, appointment date, and contact number through a form. Validate the input and display an appointment-details confirmation.

Week 7(b), Exercise 1 - Cook profile CRUD. Design and implement a database-driven ChefKart Cook Information Management System using Flask, SQLite, and HTML forms. Provide Create, Read, Update, and Delete operations for profiles containing name, cuisine, city, visit fee, and availability.

Week 7(b), Exercise 2 - Kitchen staff records. Design a Python web application for ChefKart Kitchen Staff Management using Flask and SQLite. Accept staff name, role, contact number, and monthly salary through a form, validate and store the details, and display the saved staff records.

The appointment exercise validates and displays details; it does not reserve a slot or save appointments. The two Week 7(b) exercises use a real SQLite database and retain their records after restarting.

All six exercises are included. The original React/Express application and the earlier Unit 4 student examples remain in the project. Full executable source is in the ZIP; the guide explains the important code in short pieces. This PDF includes the earlier Unit 4 guide as an appendix.

2. Run the exercises

Week 6: MongoDB and Node.js

Install Node.js 20 or later, MongoDB Community Server, and MongoDB Shell (`mongosh`). Start MongoDB Server before running either example. Compass is a database viewer; installing Compass alone does not start a database server.

From the extracted ChefKart root, run:

```
npm install
cd coursework/week6
mongosh "mongodb://127.0.0.1:27017/chefkart_week6" --file service-catalog.mongosh.js
cd ../..
npm run week6:node
```

Both programs use the separate database `chefkart_week6`. The shell file selects that database explicitly. For Node.js, an optional `WEEK6_MONGODB_URI` environment variable can override the default URI. Keep coursework samples in a separate database.

If PowerShell blocks `npm.ps1`, use `npm.cmd install` and `npm.cmd run week6:node`, or use CMD.

Week 7: Python, Flask and SQLite

Use Python 3.10 or later. From the same extracted ChefKart root, run:

```
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r coursework/week7/requirements.txt
.\.venv\Scripts\python.exe coursework/week7/app.py
```

Open **http://127.0.0.1:5001**. Keep the terminal open; press Ctrl+C to stop. The commands call the virtual environment directly, so activation is unnecessary. If Windows has no `py` launcher, use `python -m venv .venv` for the first command.

On macOS or Linux, use `python3 -m venv .venv`, then `.venv/bin/python -m pip install -r coursework/week7/requirements.txt` and `.venv/bin/python coursework/week7/app.py`.

SQLite comes with Python. The application creates its tables automatically in `coursework/week7/instance/chefkart.sqlite3`. MongoDB is not needed for Week 7. This separate Flask practice app has no login and is intended to run locally with sample data.

Existing ChefKart website

After `npm install`, use `npm run dev` for the JSON-backed website, or `npm run dev:mongo` with MongoDB running. Open **http://127.0.0.1:3000**. Its records are separate from the Week 6 and Week 7 examples.

3. Simple code structure

The following paths are relative to the `coursework/` folder.

File or folder	Purpose
`AIMS.md`	The six corrected aims, ready to use in your practical record.
`week6/service-catalog.mongosh.js`	MongoDB shell commands for cooking services.
`week6/chef-crud.cjs`	One Node.js console program with connection, schema and CRUD.
`week7/app.py`	Flask routes: receive requests, validate, query and render pages.
`week7/validation.py`	Small reusable functions for text, date, phone and money checks.
`week7/db.py`	Open and close SQLite connections; create tables.
`week7/schema.sql`	The chefs and employees table definitions.
`week7/templates/`	HTML pages and shared form fields.
`week7/static/style.css`	Responsive ChefKart colours, spacing and forms.
`week7/tests/test_app.py`	Thirteen tests using temporary databases.

Follow one form from start to finish

1. A browser GET request opens a route such as `/chefs/new`.
2. Flask uses `render_template()` to display the HTML form.
3. The browser sends the completed form as a POST request.
4. The route reads `request.form` and calls a validation function.
5. If input is invalid, the same form displays errors and keeps the entered values.
6. If input is valid, Week 7(a) displays a confirmation; Week 7(b) executes SQL and calls `commit()`.
7. After saving, Flask redirects to the records page, which reads the saved rows with SELECT.

`create_app()` builds the application and creates missing tables. `if __name__ == "__main__"` starts the server only when you run `app.py` directly. This also lets tests create a separate app with a temporary database.

The HTML files use Jinja, Flask's template language. `{{ value }}` displays a value; `{% for ... %}` repeats a block for each record. `base.html` supplies common navigation, while `_fields.html` supplies reusable labelled fields.

4. Week 6(a): MongoDB commands

File: `coursework/week6/service-catalog.mongosh.js`

The library example becomes a cooking service catalogue. A book ID becomes a service ID; the title becomes a service title; the author becomes a chef name. Category, price and availability retain their meaning in the ChefKart theme.

MongoDB field	Example and meaning
`serviceId`	Generated run ID plus S01; unique service identifier.
`title`	Daily home cooking; service description.
`chefName`	Meera Sharma; the cook providing the service.
`category`	Monthly, One-time or Party.
`price`	12974; numeric price in INR for that service category.
`available`	Boolean true or false.
`demoRun`	Generated tag used to select only this run's sample records.

The four operations

The full file uses `insertMany()` to insert three complete service documents. Its two initially available services have IDs ending in S01 and S03.

```
// READ: retrieve the available services inserted by this run.
services.find({ demoRun: runId, available: true }).toArray();

// UPDATE: change availability for one selected service.
services.updateOne(
  { serviceId: `${runId}-S02` },
  { $set: { available: true } }
);

// DELETE: remove the selected party service.
services.deleteOne({ serviceId: `${runId}-S03` });
```

`$set` changes the named field without replacing the complete document. A unique index on `serviceId` prevents duplicate IDs. Every execution generates a fresh `runId`, so repeating the file adds a new sample set without deleting unrelated records.

Expected result

The terminal shows three inserted services, two initially available services, one successful availability update and one deletion. **Two records remain from that run.** In Compass, open `chefkart_week6` and the `cooking_services` collection. Use the printed run ID in the filter `{ "demoRun": "paste-the-run-id" }` to inspect that sample set.

Run this file with **mongosh**, not with Node.js. Commands such as `db.getSiblingDB()` and `printjson()` belong to the MongoDB shell.

5. Week 6(b): Node.js connection and CRUD

File: `coursework/week6/chef-crud.cjs`

The student collection becomes a cook collection named `chefs`. Mongoose is the Node.js library used to describe the fields and communicate with MongoDB. The existing project already supplies this dependency.

Read the program in order

Code	Simple explanation
`new mongoose.Schema(...)`	Defines chef ID, name, cuisine, city, price and availability.
`mongoose.model(..., "chefs")`	Creates a model connected to the chefs collection.
`await mongoose.connect(uri)`	Waits for the database connection before running CRUD.
`await Chef.init()`	Waits for the unique chef ID index.
`await Chef.create(...)`	Inserts the sample cook.
`await Chef.find({ chefId }).lean()`	Retrieves matching records as plain JavaScript objects.
`await Chef.findOneAndUpdate(...)`	Updates the matching cook and returns the new values.
`await Chef.deleteOne({ chefId })`	Deletes only the cook inserted by the current run.
`finally { await mongoose.disconnect(); }`	Closes the database connection even if an error occurs.

Understand the update

```
const updated = await Chef.findOneAndUpdate(
  { chefId },
  { $set: { price: 549, available: false } },
  { new: true, runValidators: true }
).lean();
```

The first object selects the record. The second object gives the changes. `new: true` returns the updated record. `runValidators: true` checks the changed values against the schema. `await` lets the database finish before the next statement runs.

Expected console sequence

```
CONNECTED: MongoDB is ready.
INSERT: cook with price 499 and available true
RETRIEVE: array containing that cook
UPDATE: same cook with price 549 and available false
DELETE count: 1
RETRIEVE after delete: null
```

The actual INSERT, RETRIEVE and UPDATE output includes the full JSON record and generated IDs. The sample cook is intentionally deleted by the end of the program to demonstrate the D in CRUD. Existing unrelated records remain unchanged.

6. Week 7(a): validate and display forms

Exercise 1: cook details

Open **http://127.0.0.1:5001/cook-form**. Enter a cook name and cuisine, for example Meera Sharma and South Indian. Choose **Review cook details**. The confirmation page displays the cleaned values.

The route is `cook_form()` in `app.py`. `validate_cook()` in `validation.py` trims both fields and requires 2 to 80 characters. The page is `templates/cook_form.html`; the output is `templates/confirmation.html`.

```
# The central decision used by each form route:
values, errors = validate_cook(request.form)
if not errors:
    # Display values on confirmation.html.
    ...
# Otherwise display the form again with its error messages.
```

This is an explanatory excerpt; the complete runnable route is in `app.py`. GET displays the blank form. POST checks the values. Invalid input returns HTTP 400 with field errors; valid input returns HTTP 200 with the confirmation.

Exercise 2: cooking appointment

Open **http://127.0.0.1:5001/appointment**. Enter a customer name, select a sample cook, choose today or a future date, and enter a ten-digit contact number. Choose **Confirm details** to see the validated details.

Field	Server validation
Customer name	Trim spaces; require 2 to 80 characters.
Preferred cook	Must match one of the three sample cook choices.
Appointment date	Must be a real YYYY-MM-DD date, today or later.
Contact number	Exactly ten digits, without spaces or a country prefix.

The route is `appointment()` and its validator is `validate_appointment()`. The choices are declared in `CHEF_CHOICES`. They are sample choices; they are not loaded from the Week 7(b) cook database. The date check uses the server's local date.

Try both a valid and invalid submission

Leave a field blank, enter an invalid phone number, or choose yesterday. The form shows errors and keeps the entered values. Correct the highlighted fields and submit again. Names and other entered text are escaped by Jinja when shown in HTML.

Week 7(a) is about form handling and validation. Neither form inserts a database record. The appointment confirmation explicitly states that it does not reserve a cooking slot.

7. Week 7(b): cook profile CRUD

Open **http://127.0.0.1:5001/chefs**. Choose **Add a cook**, enter the fields and select **Save cook profile**. The records page displays saved profiles, each with Edit and Delete controls.

Operation	Flask route	Database statement
Create	GET/POST `/chefs/new`	INSERT INTO chefs
Read	GET `/chefs`	SELECT * FROM chefs
Update	GET/POST `/chefs/<id>/edit`	UPDATE chefs ... WHERE id = ?
Delete	POST `/chefs/<id>/delete`	DELETE FROM chefs WHERE id = ?

GET on the edit route loads the current values. POST validates and updates that record. Delete asks for a browser confirmation and submits a POST form. A missing profile returns HTTP 404; a GET request cannot delete a record.

Create a record with bound values

```
db = get_db()
db.execute(
    "INSERT INTO chefs (name, cuisine, city, fee_paise, available) "
    "VALUES (?, ?, ?, ?, ?)",
    (values["name"], values["cuisine"], values["city"],
     values["fee_paise"], values["available"]),
)
db.commit()
```

Each `?` is a placeholder. SQLite receives the values separately from the SQL statement. `commit()` saves the change; without it, a write may be lost when the connection closes. The update and delete operations also use bound values.

Fields and validation

Name, cuisine and city each require 2 to 80 characters. Visit fee must be from INR 100 to INR 100,000 with at most two decimal places. Availability must be Available or Unavailable. SQLite generates the integer profile ID automatically.

The database stores fees as whole **paise**: INR 499.95 becomes 49995. `money_value()` uses Decimal to avoid floating-point rounding, and the `rupees` template filter formats the value for display. SQLite stores availability as 1 or 0.

Verify the complete cycle

Create Meera Sharma at INR 499.95 and Available. Confirm she appears in the list. Edit the fee to INR 549.05 and availability to Unavailable. Restart the server and check the saved changes. Delete the profile and confirm it disappears. Records entered on this page are separate from the existing React website.

8. Week 7(b): kitchen staff database

Open **http://127.0.0.1:5001/employees**. Enter the details and choose **Save staff record**. The page shows the saved staff on the right on wider screens, or below the form on a phone.

Field	Example and validation
Staff name	Ravi Kumar; 2 to 80 characters.
Role	Cook, Kitchen assistant, Kitchen coordinator or Customer support.
Contact number	9876543210; exactly ten digits.
Monthly salary	INR 18,000.50; enter 18000.50, without commas. Allowed range: 1 to 1,000,000.

How saving works

The `employees()` route handles both GET and POST. A POST calls `validate_employee()`. Valid values are inserted into the `employees` table, committed, and followed by a redirect. The next GET selects the saved rows and passes them to `employees.html`.

```
db.execute(
    "INSERT INTO employees (name, role, phone, salary_paise) "
    "VALUES (?, ?, ?, ?)",
    (values["name"], values["role"], values["phone"],
     values["salary_paise"]),
)
db.commit()
```

Salary is stored as integer paise, just like a cook's visit fee. Phone numbers are stored as text. The database creates the staff ID; the user does not type one.

Expected result

After saving Ravi Kumar as a Kitchen assistant with phone 9876543210 and salary 18000.50, the saved record displays **INR 18,000.50**. Refreshing the page does not insert a duplicate. Restarting the app does not remove the record.

This exercise requires accepting, storing and displaying staff records. The separate cook-profile exercise supplies full CRUD. Re-submitting the staff form deliberately creates another staff record; no automatic duplicate detection is claimed.

9. Tests and troubleshooting

Verification completed

Check	Result
Week 6 shell against MongoDB 7.0.14	Inserted three, retrieved available services, updated one, deleted one; two remained.
Week 6 Node.js against MongoDB 7.0.14	Connected, inserted, retrieved, updated and deleted the sample cook.
Existing unrelated MongoDB records	Sentinel records remained after both programs.
Flask automated checks	All 13 tests passed, including validation, CRUD, escaping and restart persistence.
Browser workflow	Both forms, cook create/edit/delete and staff save/display passed in Chromium.
Responsive layout	Six pages checked at 1280px and 375px; no horizontal overflow.

Text evidence is in `docs/verification/week6-results.txt`, `week7-results.txt` and `week7-browser-results.txt`. Verification used temporary databases and sample values. Existing Unit 4 verification is retained separately.

To repeat the Flask tests from the ChefKart root on Windows:

```
cd coursework/week7
..\..\.venv\Scripts\python.exe -m unittest discover -s tests -v
```

Return to the root afterwards using `cd ../...` The tests create and remove their own temporary databases; they do not edit your saved records.

Common fixes

Symptom	What to do
MongoDB connection refused	Start MongoDB Server; check the URI and port.
mongosh is not recognised	Install MongoDB Shell and reopen the terminal.
No module named flask	Install requirements with the same .venv Python used to run app.py.
Port 5001 already in use	Stop the earlier Flask process, or set CHEFKART_FLASK_PORT to another free port.
Form says reload and submit again	Reload the page after restarting Flask; its local session key changes on restart.
No cook remains after Week 6(b)	Expected: its final operation deletes the sample it created.
Flask records do not appear in React	Expected: these independent exercises use different databases.

Form submissions include a session token to prevent cross-site form submissions. Table creation uses `CREATE TABLE IF NOT EXISTS`, so startup does not clear records. To back up Week 7 data, stop the server and copy the SQLite file.

10. Practical record and reference

For each exercise, use this simple order in your submission: **Aim, files used, steps, relevant code, observed output, result**. The exact aims are also available in `coursework/AIMS.md`.

Results you can demonstrate

- Week 6(a): MongoDB cooking service insert, available-service query, availability update and deletion.
- Week 6(b): Node.js connection followed by cook collection CRUD and a clean disconnect.
- Week 7(a)-1: cook name and cuisine checked and displayed.
- Week 7(a)-2: cooking appointment details checked and displayed in a confirmation.
- Week 7(b)-1: cook profiles created, read, updated, deleted and persisted in SQLite.
- Week 7(b)-2: kitchen staff details accepted, stored and displayed from SQLite.

Which database belongs to which program?

Program	Default data location
Week 6 shell and Node.js	MongoDB: chefkart_week6
Week 7 Flask app	coursework/week7/instance/chefkart.sqlite3
Existing ChefKart website, JSON mode	backend/data/database.json
Existing ChefKart website, MongoDB mode	MongoDB: chefkart_unit4

Switching between programs does not copy records. The Week 6 shell leaves two services per run; the Node.js console example removes its own sample cook. Week 7 starts with empty tables and keeps records that you submit.

Official learning references

Flask routing and form handling: https://flask.palletsprojects.com/en/stable/quickstart/

Flask SQLite tutorial: https://flask.palletsprojects.com/en/stable/tutorial/database/

The following appendix preserves the earlier Unit 4 guide, including its MongoDB/Mongoose explanations and student reference exercises. Use the ChefKart aims and code in the first part of this guide for the newly requested Week 6 and Week 7 work.

WAF Unit 4 - completed practical guide

MongoDB, Mongoose, Student CRUD and integration into ChefKart

Based on the supplied 52-slide WAF-UNIT-4-dated-06-10-2026 presentation. The upload is a PowerPoint file even though the request refers to a PDF. This guide covers all practical examples and explains the introductory topics. Full runnable code is in the project ZIP.

1. What was completed

Slides	Topic	Completed material
1-11	NoSQL, types, terminology, document structure and features	Section 2 below
12-14	Windows installation and connection	Section 3 and unit4/README.md
15-23	Shell CRUD and seven query operators	unit4/mongodb-shell/crud-demo.js
24-29	Mongoose, schemas, models and validation	Sections 5-6; both Student model files
30-35	First Express connection and POST with CGPA	unit4/connectivity-demo/
36-46	Complete Student CRUD API	unit4/student-crud/
47-52	Thunder Client POST, GET, PUT and DELETE	unit4/THUNDER_CLIENT.md and requests.http
Application to supplied project	Mongoose-backed ChefKart CRUD	backend/models/ and backend/services/store.js

The slide deck is a teaching sequence rather than a numbered assignment sheet. Nothing in it asks for payment processing or a deployed website. The examples here use actual MongoDB for the database practicals.

Read the small Student lab first. Then read the ChefKart models to see how the same ideas work in a larger application. Comments describe the purpose of functions and important blocks. The original frontend/backend walkthrough is in CODE_WALKTHROUGH.md.

2. NoSQL concepts in simple language

NoSQL means Not Only SQL. These databases use models such as documents, key-value pairs, wide-column records and graphs. Choose a model based on the data and access patterns. A flexible schema does not mean data should have no rules.

Type	Data arrangement	Examples from the slides
Document	Objects containing named fields and nested values	MongoDB, CouchDB
Key-value	One value looked up by a key	Redis, DynamoDB
Column-family	Related columns grouped within records	Cassandra, HBase
Graph	Nodes connected through relationships	Neo4j, Neptune

MongoDB is a document database. Applications often show its documents as JSON-like objects. MongoDB stores them internally as BSON, which supports additional types such as dates and ObjectIds.

Relational term	MongoDB term
Database	Database
Table	Collection
Row/record	Document
Column	Field
Primary key	_id

A database contains collections. A collection contains documents. A document contains fields. For example, `studentDB` contains `students`, and one student document contains `name`, `rollNo`, `branch` and `marks`.

SQL databases	MongoDB/document databases
Organize records into tables and rows	Organize records into collections and documents
Usually declare columns and constraints	Can store varying document shapes; application/database validation can impose rules
Often use joins and foreign keys	Often embed related data or store references
Use SQL	Use database-specific APIs and query syntax

The slides' scaling comparison is a simplified tendency, not an absolute rule. Relational systems can scale horizontally, and document databases can also scale vertically. SQL and NoSQL systems both support useful relationships and consistency mechanisms.

Other features named in the slides: indexes can speed up suitable queries; aggregation processes documents through stages to calculate summaries; replica sets maintain copies for availability; sharding partitions data across servers. The class practical uses a single MongoDB server and does not configure a replica set or sharded cluster.

3. Install and run on Windows

Install Node.js with npm, VS Code, MongoDB Community Server, MongoDB Compass and MongoDB Shell (`mongosh`). Pick a MongoDB release supported by your Windows version. In the Windows MongoDB installer, use Complete installation and install MongoDB as a service. Compass is a viewer, while the server actually stores and serves data. `mongosh` may require a separate installation.

In Compass, create a connection using:

```
mongodb://127.0.0.1:27017
```

Extract the ZIP. Open the extracted ChefKart folder in VS Code and its terminal. Run:

```
npm install
npm run unit4
```

Expected startup messages:

```
MongoDB Connected Successfully
Student API: http://127.0.0.1:3001
```

Keep the terminal open and send API requests through Thunder Client. The Student API deliberately uses port 3001 so it does not conflict with ChefKart's frontend on 3000. The connection-only exercise uses 3002. ChefKart's backend remains on 8000.

For ChefKart with MongoDB, open another root terminal and run `npm run dev:mongo`. Open `http://127.0.0.1:3000`. For an administrator, stop the app, run `npm run admin:mongo`, enter a new account, restart and sign in.

PowerShell accepts `cd "C:\path\to\ChefKart"`. CMD accepts `cd /d "C:\path\to\ChefKart"`. Replace the example path with your extracted folder. If PowerShell blocks npm.ps1, use `npm.cmd` in place of `npm`.

4. Shell CRUD and query operators

The shell and Node.js run different kinds of JavaScript environments. Run the shell file with **mongosh**, not `node`:

```
mongosh "mongodb://127.0.0.1:27017" --file unit4/mongodb-shell/crud-demo.js
```

It selects `waf_unit4_practice`, checks that its students collection is empty, inserts four records, demonstrates every operation, and deletes those practice records. Application databases remain separate.

In an interactive mongosh session, the corrected selection commands are:

```
use college
db.createCollection("students")
```

`use` is lowercase and selects a **database**. `Use students` in slide 18 is incorrect. You do not use `use` to select a collection. `db.students` accesses the collection; the first insert can create it automatically, so explicit createCollection is optional. In a script file, use `db = db.getSiblingDB("college")` instead of the interactive `use` helper.

Create

```
db.students.insertOne({
  name: "Rahul", rollNo: "CSE101", branch: "CSE", marks: 85
});
db.students.insertMany([
  { name: "Priya", rollNo: "CSE102", branch: "CSE", marks: 92 },
  { name: "Arjun", rollNo: "CSE103", branch: "CSE", marks: 76 },
  { name: "Meera", rollNo: "ECE101", branch: "ECE", marks: 80 }
]);
```

The script inserts Rahul once. Copying slides 18 and 19 consecutively would insert Rahul twice because that shell example has no unique roll-number index.

Read

```
db.students.find();
db.students.find().pretty();
db.students.findOne({ rollNo: "CSE101" });
db.students.find({ marks: { $gt: 80 } });
db.students.find({ branch: "CSE" });
```

`find()` returns a cursor over matching documents. `findOne()` returns one document or null. The runnable script converts cursors to arrays for clear printed output. `pretty()` changes display, not the stored data.

Operator/filter before updates	Meaning	Matching students
`{ marks: { $eq: 85 } }`	Equal to 85	Rahul
`{ marks: { $gt: 80 } }`	Greater than 80	Rahul, Priya
`{ marks: { $gte: 80 } }`	At least 80	Rahul, Priya, Meera
`{ marks: { $lt: 80 } }`	Less than 80	Arjun
`{ marks: { $lte: 80 } }`	At most 80	Arjun, Meera
`{ branch: { $ne: "CSE" } }`	Not CSE	Meera
`{ branch: { $in: ["CSE", "ECE"] } }`	In either listed branch	All four

Update

```
db.students.updateOne(
  { rollNo: "CSE101" }, { $set: { marks: 90 } }
);
db.students.updateMany(
  { branch: "CSE" }, { $set: { department: "Computer Science" } }
);
db.students.updateOne(
  { rollNo: "CSE101" }, { $inc: { marks: 5 } }
);
```

The first argument selects records. The second describes changes. `$set` assigns a value; `$inc` adds to a number. Rahul starts at 85, becomes 90 and ends at **95**. The department field is added to all three CSE documents.

Delete

```
db.students.deleteOne({ rollNo: "CSE101" });
db.students.deleteMany({ branch: "CSE" });
db.students.deleteMany({});
```

In this sequence, the deleted counts are **1, 2, 1**. Rahul is removed first, Priya and Arjun next, then Meera. The final count is **0**. An empty filter matches every document, so the complete script confines this example to its dedicated practice database.

5. First Express and Mongoose example

Mongoose is an Object Data Modeling library. A schema describes fields and validation. A model uses that schema to read and write documents. Express handles HTTP requests. MongoDB stores the documents.

The first example from slides 28-35 is in `unit4/connectivity-demo/`. Its schema uses a **text** rollNo such as `23CSE101`, and **cgpa** between 0 and 10. Start it with:

```
npm run unit4:connect
```

Send POST to `http://127.0.0.1:3002/students` with:

```
{
  "name": "Ravi Kumar",
  "rollNo": "23CSE101",
  "branch": "CSE",
  "cgpa": 8.7
}
```

The route picks these four fields from req.body, creates `new Student(...)`, then calls `await student.save()`. It returns status 201 and the saved document. Compass shows it in `collegeDB.students`.

`mongoose.connect()` must succeed before the HTTP server starts. `await` pauses that async function until the operation finishes. A connection error produces a visible message instead of claiming the database is ready.

The second example is a separate lab. It uses numeric rollNo, marks from 0 to 100, database studentDB and port 3001. Keeping both examples avoids mixing the two schemas used by the slides.

6. Understand the Student model

The complete file is `unit4/student-crud/models/Student.js`.

Field/rule	Explanation
name: String, required, trim	Requires a non-empty name and removes edge spaces
rollNo: Number, required, unique	Requires a roll number and creates a unique index
rollNo min 1, Number.isInteger	Rejects zero, negatives and fractions
branch: String, required	Requires a non-empty branch
marks: Number, required, min 0, max 100	Enforces the allowed mark range
mongoose.model("Student", studentSchema)	Creates the Student model; default collection name is students

MongoDB gives the document an `_id`; Mongoose normally uses an ObjectId for this schema. Its string representation contains **24 hexadecimal characters**. Copy the returned ID. The shortened example IDs in the slides are illustrations, not valid IDs to reuse.

`unique: true` is an index instruction rather than a normal Mongoose validator. Startup waits for `Student.init()` so the index is ready. Error code 11000 is converted to HTTP **409 Conflict**. Field validation errors produce **400 Bad Request**.

JSON numbers must not be in quotation marks in this API. Use `"rollNo": 101` and `"marks": 85`. The initial CGPA example intentionally uses `"rollNo": "23CSE101"` instead.

7. Understand the Student server

The complete file is `unit4/student-crud/server.js`. The five routes are kept together so a beginner can read the whole request flow in one place.

Code or function	Purpose
require("express") and require("mongoose")	Load the two installed packages
express()	Creates the web application
express.json()	Parses an incoming JSON body into req.body
app.param("id", ...)	Rejects malformed IDs before a database lookup
validateBody	Allows only known fields and expected JSON types
app.post/get/put/delete	Connects an HTTP method and URL to a function
try/catch and next(error)	Passes failed operations to the common error handler
app.use((error, req, res, next) => ...)	Formats readable JSON errors
start()	Connects MongoDB, waits for the index, starts HTTP and handles shutdown
require.main === module	Starts when run directly; permits importing app in tests

`req` is the request, `res` sends the response, and `next()` continues to the next matching middleware. `next(error)` switches to the error handler. `req.body` holds submitted JSON. `req.params.id` holds the ID from the URL. A response such as `res.status(201).json(student)` sends a status code and a JSON document.

Operation	HTTP request	Mongoose method	Success
Create	POST /students	Student.create(req.body)	201
Read all	GET /students	Student.find()	200
Read one	GET /students/:id	Student.findById(id)	200
Update	PUT /students/:id	Student.findByIdAndUpdate(...)	200
Delete	DELETE /students/:id	Student.findByIdAndDelete(id)	200

For update, the important code is:

```
const student = await Student.findByIdAndUpdate(
  req.params.id,
  { $set: req.body },
  { new: true, runValidators: true }
);
```

`new: true` returns the updated document. `runValidators: true` checks changed fields, including the marks range. Without it, update validation is not automatically enabled. A valid but missing ID produces null, which the route changes into a 404 response.

This lab follows the slides' partial update with PUT. Only submitted fields change. A production API might choose PATCH for partial updates, but the learning exercise keeps the exact method requested by the slides.

8. Thunder Client demonstration

Start `npm run unit4`. Install/open Thunder Client in VS Code, choose New Request, and use the following sequence. Select Body -> JSON for POST and PUT.

1. POST `http://127.0.0.1:3001/students` with the JSON below. Expect 201 and copy `_id`.
2. GET the same URL. Expect 200 and an array containing Rahul.
3. GET `http://127.0.0.1:3001/students/YOUR_ID`. Expect 200 and one document.
4. PUT to that ID URL with `{ "marks": 92 }`. Expect 200 and updated marks.
5. GET again. Confirm the persisted value is 92.
6. DELETE that ID URL. Expect 200 and the deletion message.
7. GET that ID again. Expect 404.

```
{
  "name": "Rahul",
  "rollNo": 101,
  "branch": "CSE",
  "marks": 85
}
```

Before deleting, also try marks 101 and a repeated roll number. The first returns 400; the second returns 409. Open Compass to inspect studentDB.students while the record exists. Save your own screenshots if your teacher requires submission evidence.

The full, copyable steps are in `unit4/THUNDER_CLIENT.md`. The `requests.http` file is a request reference; sending it directly requires VS Code REST Client. Thunder Client can use the same URLs and bodies through its normal request editor.

9. How the same work is applied to ChefKart

The supplied ChefKart app already had a React frontend and Express CRUD routes. Its old MongoDB adapter used raw collections through Mongoose's connection. The update adds explicit Mongoose schemas and models, then uses model methods in the existing store. This makes schema/model/validation/CRUD visible in the actual project.

New file in backend/models	Collection or responsibility
User.js	Account names, emails, password hashes, roles; unique email
Chef.js	Cook profiles, prices, cuisine lists and contact fields; unique owner
Booking.js	Customer/cook references, date, service, guests, estimate and status
Lead.js	Private contact, career and investor enquiries
Content.js	Shared content schema for the eleven allowed content collections
Metadata.js	Remembers whether starter content has been inserted
common.js	Shared string ID and timestamp fields
index.js	Maps the allowed collection names to their models

ChefKart retains its existing string IDs and ISO timestamp strings so the frontend and existing v2 records keep the same field formats. The standalone Student lab demonstrates MongoDB ObjectIds. A schema's `ref` records a relationship for Mongoose; it does not automatically enforce a database foreign key. Controllers check existence and permissions.

`backend/services/store.js` now uses `Model.create()`, `find().lean()`, `findById().lean()`, `findByIdAndUpdate()` and `findByIdAndDelete()` in MongoDB mode. `lean()` returns plain JavaScript objects that the existing controllers expect. The update method enables validators. Index initialization finishes before startup accepts requests.

The same cook cannot have two active bookings at the same exact start time. A unique partial index checks only records whose status is booked. Cancelling a booking releases that exact slot. This does not calculate duration overlaps or reserve a repeating monthly calendar.

Follow one booking through the code

1. React's BookingForm collects service, date, address, guests and notes.
2. The shared frontend API helper sends POST /api/bookings with the login token.
3. Express parses JSON. requireAuth verifies the token and loads the user.
4. bookingController validates fields and finds the cook.
5. The controller calculates the price estimate and selects the owner/status itself.
6. store.create chooses the Booking model in MongoDB mode.
7. Mongoose validates the document. MongoDB inserts it and enforces the unique index.
8. Express returns 201. React displays the saved booking under My bookings.

Read CODE_WALKTHROUGH.md for every existing controller, middleware, component and page. API.md lists the ChefKart routes. The new scripts/mongo.mjs ensures dev:mongo, start:mongo and admin:mongo use the same configured MongoDB database.

10. Run and check the completed project

Purpose	Root command
Install all workspace dependencies	npm install
Student API	npm run unit4
Initial connection exercise	npm run unit4:connect
ChefKart with MongoDB	npm run dev:mongo
Create MongoDB administrator	npm run admin:mongo
ChefKart local JSON demo	npm run dev
Build frontend	npm run build
Serve built app with MongoDB on port 8000	npm run start:mongo
Check local ChefKart behaviour	npm test
Check ChefKart against MongoDB	npm run test:mongo
Check both student examples against MongoDB	npm run test:unit4

Use MongoDB mode for the assignment. Open `http://127.0.0.1:8000/api/health` and check for `storage: "mongodb"`. The local JSON demo is a separate convenience option; it does not demonstrate MongoDB persistence. Changing modes does not migrate records between them.

The automated tests use new test databases with random names and remove those databases after the run. They cover create/read/update/delete, validation and missing records. ChefKart additionally checks authentication, ownership, private data, content management and slot conflicts. See VERIFICATION.md for the checks actually completed for this revision.

11. Common errors and viva answers

Symptom	Fix
`Use students` syntax error	Use lowercase `use college`; access records through db.students
mongosh is not recognized	Install MongoDB Shell separately, add its bin folder to PATH, reopen terminal
Connection refused on 27017	Start MongoDB Server; connecting Compass alone does not start it
npm cannot find package.json	Run commands in the extracted ChefKart root
EADDRINUSE	Stop the older process using that port, or set STUDENT_PORT/CONNECTIVITY_PORT
POST returns duplicate rollNo	Use another number or delete the previous record
An ID request returns 400	Copy the real 24-character _id returned by POST
ChefKart works but Compass shows no records	Check health storage and start dev:mongo for database work
No admin link	Create an admin in the same storage mode, then sign in with it

Why use a schema? To define field types and validation rules. **Why use a model?** It provides methods to work with the collection. **Why express.json()?** HTTP bodies arrive as bytes; the middleware parses JSON into req.body. **Why async/await?** Database operations finish asynchronously. **Why validate on the server?** A caller can send HTTP requests without using your form. **What does 201 mean?** Created. **400?** Invalid request. **404?** No matching route/record. **409?** A conflict such as a duplicate roll number.

What is middleware? Code that runs during request processing before a response, or in the error path. Express middleware and Mongoose middleware are different: Express handles HTTP requests; Mongoose hooks run around model operations. Mongoose runs validation before save by default. The models here need no extra custom pre-save hook because password hashing remains explicit in ChefKart's auth controller.

12. Source references

Primary course source: the uploaded WAF Unit 4 presentation dated 06-10-2026, slides 1-52. The runnable examples correct its shell capitalization, collection selection, duplicated insertion sequence and illustrative short IDs.

- MongoDB Windows installation: https://www.mongodb.com/docs/v8.0/tutorial/install-mongodb-on-windows/
- MongoDB Shell download: https://www.mongodb.com/try/download/shell
- Mongoose 8 validation: https://mongoosejs.com/docs/8.x/docs/validation.html
- Mongoose update behaviour: https://mongoosejs.com/docs/tutorials/findoneandupdate.html
- Thunder Client: https://docs.thunderclient.com/get-started

The ZIP includes the complete commented source. Installation on your Windows computer and screenshots taken in your own Thunder Client session must be performed there. The verification report distinguishes executed checks from those local steps.